

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Аппаратно-программное обеспечение ЭВМ и сетей»
Тема: «Изучение аппаратно-программной
архитектуры процессоров семейства Intel – разработка
ассемблерной вставки»

Выполнил:
Студент 4-го курса
Группы АС-66
Езепчук А. С.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками

Ход работы

Задания:

1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).
2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.
3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов. В отчете отразить выводы.

Содержание отчета:

- титульный лист;
- описание варианта задания;
- текст программы;
- скриншоты результатов.

Индивидуальное задание:

6. Нечетные символы заменить на символ +

Код:

```
#include <iostream>
#include <string>
#include <chrono>

using namespace std;
using namespace chrono;
string asmFunction(const string& a)
{
    string result = a;

    const size_t len = result.size();
    char* str = result.data();
    size_t i = 0;

    while (i < len)
    {
        asm volatile(
            "movb $43, (%[str], %[i], 1)\n\t"
            :
            : [str] "r" (str),
              [i] "r" (i)
            : "memory"
        );

        i += 2;
    }
}
```

```

    }

    return result;
}
string cppFunction(const string& a)
{
    string result = a;

    for (size_t i = 0; i < result.size(); i += 2)
    {
        result[i] = '+';
    }

    return result;
}

int main()
{
    string a = "ABCDEF";
    string resultCpp = cppFunction(a);
    string resultAsm = asmFunction(a);

    cout << "Original string: " << a << endl;

    cout << "\nC++ result: " << resultCpp << endl;
    cout << "ASM result: " << resultAsm << endl;

    if (resultCpp == resultAsm)
        cout << "Results are identical." << endl;
    else
        cout << "ERROR: results are different!" << endl;

    const int N = 1000000;

    string temp;
    temp = cppFunction(a);
    temp = asmFunction(a);
    auto startCpp = high_resolution_clock::now();

    for (int i = 0; i < N; ++i)
    {
        temp = cppFunction(a);
    }

    auto finishCpp = high_resolution_clock::now();

    auto timeCpp =
        duration_cast<nanoseconds>(finishCpp - startCpp).count();
    auto startAsm = high_resolution_clock::now();

    for (int i = 0; i < N; ++i)
    {
        temp = asmFunction(a);
    }

    auto finishAsm = high_resolution_clock::now();

    auto timeAsm =
        duration_cast<nanoseconds>(finishAsm - startAsm).count();
    cout << "\nNumber of calls: " << N << endl;

    cout << "\nC++ total time: "
        << timeCpp << " ns" << endl;

    cout << "C++ average: "
        << static_cast<double>(timeCpp) / N
        << " ns" << endl;

    cout << "\nASM total time: "
        << timeAsm << " ns" << endl;
}

```

```

cout << "ASM average:      "
      << static_cast<double>(timeAsm) / N
      << " ns" << endl;
if (timeAsm < timeCpp)
{
    double percent =
        100.0 * (1.0 - static_cast<double>(timeAsm) / timeCpp);

    cout << "\nASM is faster by approximately "
          << percent << "%" << endl;
}
else if (timeCpp < timeAsm)
{
    double percent =
        100.0 * (1.0 - static_cast<double>(timeCpp) / timeAsm);

    cout << "\nC++ is faster by approximately "
          << percent << "%" << endl;
}
else
{
    cout << "\nExecution times are approximately equal." << endl;
}

return 0;
}

```

```

PS C:\Users\VivoBook\Desktop\AB05A\APO\APOEVMiS_AS66\reports\ezepchuk\lab1\src> ./lab1
Original string: ABCDEF

C++ result:  +B+D+F
ASM result:  +B+D+F
Results are identical.

Number of calls: 1000000

C++ total time:  26015600 ns
C++ average:    26.0156 ns

ASM total time:  15000100 ns
ASM average:     15.0001 ns

ASM is faster by approximately 42.3419%.

```